Nicolas
Dodart

22409199
M1 INFO-IA-SD-S

Rapport - Projet conception, interactions Humain-Machine

Table des matières

1. Introduction 2
2. Vie du projet 2
3. Diagrammes et organisation des classes 3
4. Design patterns utilisés 3
5. Algorithmes et logique du projet 5
7. Améliorations possibles 7
ANNEXE 8

Nicolas 22409199
Dodart M1 INFO-IA-SD-S

1. Introduction

Ce projet consiste à développer une application graphique permettant à un joueur de reproduire un ensemble de formes géométriques. L'objectif est de maximiser la surface occupée par ses propres formes tout en évitant les collisions avec des formes de référence affichées à l'écran.

Le joueur peut créer différentes formes (cercles, rectangles), les déplacer, les redimensionner et les supprimer. L'application propose également plusieurs modes de jeu, dont un mode aléatoire, un mode basé sur un fichier et un mode deux joueurs.

Le projet met en œuvre plusieurs concepts importants de programmation orientée objet, notamment l'utilisation de design patterns afin de structurer le code.

- Dans la partie « Vie du projet », je présente de manière simple le déroulement du projet ainsi que mon expérience personnelle. Le reste de ce rapport aura pour objectif d'adopter une approche plus scientifique et formelle.

2. Vie du projet

J'ai dû réaliser ce projet seul à cause de mes absences. L'ayant déjà fait l'année dernière dans cette classe, cela m'a permis de me rendre compte des bons et des mauvais côtés du travail seul et en groupe.

Malgré le fait d'avoir fait un diagramme de classes quasiment complet dès le début du projet, je me suis rendu compte que l'ajout d'une fonctionnalité « non prévue » peut changer beaucoup de choses. Il est donc important d'avoir une structure robuste, au risque de devoir recoder beaucoup de classes.

Pour le score, j'ai opté pour un pourcentage de remplissage de l'écran, normalisé sur 100. La pénalité appliquée, si deux formes de couleurs différentes se superposent, est d'ignorer la superficie remplie de cette forme. Même si ce n'est pas implémenté, dans le cas de formes qui disparaissent après un certain temps, cela met un grand accent sur le fait de prendre des risques ou non.

J'ai laissé le score en direct affiché à l'écran. Cela m'a permis de tester à quel point mes systèmes de collision étaient performants.

J'ai décidé de laisser le « score en direct » d'affiché sur l'écran, il m'a permis de tester mes systèmes de collision.

Je n'ai pas rencontré de difficulté majeure lors de ce projet. La principale contrainte était surtout le temps nécessaire pour mettre les différentes fonctionnalités en place correctement.

Plusieurs améliorations auraient pu être faites durant ce projet. Par exemple, le score aurait pu être un peu plus arrondi pour être plus lisible. J'ai également codé la sérialisation des données, mais le mode de jeu associé n'est pas encore très abouti.

Je n'ai pas implémenté de joueur automatique (IA), même si c'était quelque chose qui m'intéressait. De même, la difficulté où les formes disparaissent avec le temps n'a pas été ajoutée. Pourtant, cela n'aurait pas été très complexe à mettre en place grâce à la structure du projet.

Je n'ai pas non plus réalisé les formes polygonales, ce qui aurait pu apporter plus de diversité.

J'ai aussi remarqué que j'utilisais encore beaucoup de terme français quand je code , ce qui peut se révéler être un problème pour la compréhension.

Malgré cela, le projet possède plusieurs points positifs. La prévisualisation des formes lors de leur création est un élément pratique. Le redimensionnement des rectangles est aussi plus intuitif, car il repose sur un changement du point d'ancrage. Enfin, un mode deux joueurs a été implémenté, ce qui ajoute un aspect intéressant au projet

3. Diagrammes et organisation des classes

*voir annexe

4. Design patterns utilisés

Strategy

Le pattern Strategy est utilisé pour rendre certains comportements interchangeables sans modifier la structure générale du jeu. Dans mon projet, il est utilisé à deux endroits principaux.

- D'abord pour le calcul du score avec l'interface ScoreStrategy et son implémentation ScoreSurfaceStrategy. La classe Jeu ne calcule pas elle-même le score. Elle délègue ce travail à une stratégie de score fournie par le mode de jeu. Cela permet de changer facilement la manière de noter une partie sans devoir modifier la logique centrale.

- Ensuite, il est utilisé pour la génération des formes de référence avec l'interface GenerationStrategy et ses implémentations GenerationAleatoireStrategy et GenerationFichierStrategy. Ce choix m'a permis d'avoir plusieurs modes de session, par exemple une session aléatoire ou une session basée sur un fichier sérialisé.

Enfin, l'interface ModeJeu joue aussi un rôle important, car les classes ModeClassique et ModeDifficile choisissent concrètement quelle stratégie de score et quelle stratégie de génération doivent être utilisées. Cela permet de centraliser les règles de chaque difficulté.

Nicolas 22409199
Dodart M1 INFO-IA-SD-S

State

Le pattern State est utilisé pour gérer les interactions utilisateur avec la souris.

L'interface EtatInteraction définit les méthodes liées aux événements souris. La classe abstraite EtatInteractionAbstrait sert de base commune. Ensuite, chaque état concret représente une action précise : EtatCreationCercle, EtatCreationRectangle, EtatDeplacement, EtatSuppression et EtatRedimensionnement.

La classe ControleurSouris contient une référence vers l'état courant dans l'attribut etatCourant. Lorsqu'un événement souri se produit, le contrôleur ne décide pas lui-même quoi faire. Il transmet simplement l'événement à l'état actif. Cela permet de changer dynamiquement le comportement de la souris selon le bouton sélectionné dans l'interface, sans surcharger le contrôleur avec de nombreux if/else/elseif.

Command

Le pattern Command est utilisé pour encapsuler les actions du joueur, principalement afin de permettre les fonctionnalités undo et redo.

L'interface Commande définit deux opérations : executer() et annuler(). Chaque action importante est représentée par une classe concrète : CommandeAjouterForme, CommandeSupprimerForme, CommandeDeplacerForme et CommandeRedimensionnerForme.

Ces commandes sont gérées par la classe HistoriqueCommandes, qui contient les piles pileUndo et pileRedo. Lorsqu'une action est effectuée, elle est enregistrée dans l'historique. Cela permet ensuite de revenir en arrière ou de réappliquer une action.

Le pattern Command est utilisé en lien direct avec les états d'interaction. Par exemple, lorsqu'un joueur termine la création d'une forme dans EtatCreationCercle ou EtatCreationRectangle, l'action finale est encapsulée dans une commande d'ajout. De la même manière, le déplacement ou le redimensionnement d'une forme peut être mémorisé sous forme de commande afin d'être annulé plus tard.

Observer

Le pattern Observer est utilisé pour synchroniser automatiquement l'interface graphique avec l'état du modèle.

Le principe est que le modèle Jeu ne dépend pas directement de l'interface graphique. Lorsqu'un changement important se produit, par exemple après l'ajout, la suppression ou le déplacement d'une forme, la classe Jeu appelle notifierObservateurs(). Le PanneauJeu, qui observe le modèle, reçoit alors cette notification et se redessine automatiquement.

Cela évite d'avoir une vue qui doit constamment interroger le modèle pour savoir si quelque chose a changé. La mise à jour devient automatique avec l'état du jeu.

5. Algorithmes et logique du projet

Calcul du score :

Le score est basé sur le principe suivant :

- On calcule la surface totale occupée par les formes du joueur
- Cette surface est limitée à la zone de la fenêtre
- Le résultat est ensuite normalisé sur 100

Cependant, plusieurs contraintes sont appliquées.

Premièrement, lorsque deux formes du joueur se superposent, la surface commune ne doit pas être comptée deux fois. Le calcul repose donc sur une approximation de l'union des surfaces. Cela permet d'éviter qu'un joueur "triche" en empilant plusieurs formes au même endroit.

Deuxièmement, si une forme du joueur entre en collision avec une forme de référence (formes rouges), elle est considérée comme invalide. Dans ce cas, la surface correspondante n'est pas ajoutée au score. Cela introduit une dimension stratégique, car le joueur doit choisir entre prendre des risques ou rester prudent.

Enfin, le score est normalisé en fonction de la surface totale de la fenêtre :

Score = (Surface valide / Surface totale de la fenêtre) × 100

Cela permet d'obtenir un score compréhensible et comparable entre différentes parties.

Gestion des collisions :

La gestion des collisions est essentielle pour garantir la cohérence du jeu.

Chaque forme implémente ses propres méthodes de détection d'intersection. Par exemple :

- un cercle vérifie la distance entre son centre et un point ou une autre forme
- un rectangle vérifie des conditions sur les coordonnées

Cette approche permet d'éviter une centralisation du code avec de nombreux cas particuliers. Chaque forme est responsable de sa propre logique.

Les collisions sont utilisées à plusieurs niveaux :

- empêcher les formes du joueur de toucher les formes de référence
- détecter les superpositions entre formes du joueur
- gérer certaines interactions lors des déplacements et redimensionnements

Contraintes géométriques :

Les formes doivent toujours rester dans la fenêtre. Pour cela, des vérifications sont effectuées lors de :

- la création
- le déplacement
- le redimensionnement

Si une opération entraîne une sortie de la zone, elle est ajustée ou bloquée.

Un cas particulier concerne le redimensionnement. Selon le point d'ancrage choisi, la forme peut s'agrandir dans différentes directions. Il est donc nécessaire de recalculer dynamiquement les dimensions tout en respectant les limites de la fenêtre.

Interaction et mise à jour en temps réel :

Le projet fonctionne de manière interactive. Lorsqu'un joueur crée ou modifie une forme, les effets sont visibles immédiatement.

Un système de prévisualisation a été mis en place lors de la création et du redimensionnement des formes. Cela permet au joueur de voir le résultat avant validation.

De plus, le score est affiché en temps réel. Cela a été initialement conçu comme un outil de test pour vérifier la validité des calculs, mais peut s'avérer utile pour la notation ou le test du projet.

7. Améliorations possibles

Plusieurs axes d'amélioration ont été identifiés au cours du développement. Le système de score pourrait être optimisé, notamment en proposant un arrondi plus pertinent, mais aussi en enlevant la surface des formes rouges au calcule de la superficie de la fenêtre, le score de 100 étant impossible pour l'instant.

Par ailleurs, bien que la sérialisation des données ait été implémentée, le mode de jeu associé reste encore peu abouti et nécessiterait des développements supplémentaires pour correspondre entièrement a ce qui avait été demandé.

L'absence d'un joueur automatique (IA) constitue également une limite du projet. Son implémentation aurait permis d'enrichir l'expérience utilisateur et d'introduire de nouveaux scénarios de test. De même, un système de difficulté basé sur la disparition progressive des formes n'a pas été intégré.

Enfin, la prise en charge de formes polygonales n'a pas été réalisée, ce qui limite la diversité des interactions possibles.

Points positifs

Malgré ces limites, le projet présente plusieurs aspects satisfaisants. La prévisualisation des formes lors de leur création améliore significativement l'ergonomie et l'expérience utilisateur.

Le redimensionnement des rectangles a également été conçu de manière plus intuitive, notamment grâce à l'utilisation d'un point d'ancrage adapté, facilitant leur manipulation.

Enfin, l'implémentation d'un mode deux joueurs constitue un apport notable, en introduisant une dimension interactive et compétitive au projet.

Nicolas
Dodart

22409199
M1 INFO-IA-SD-S

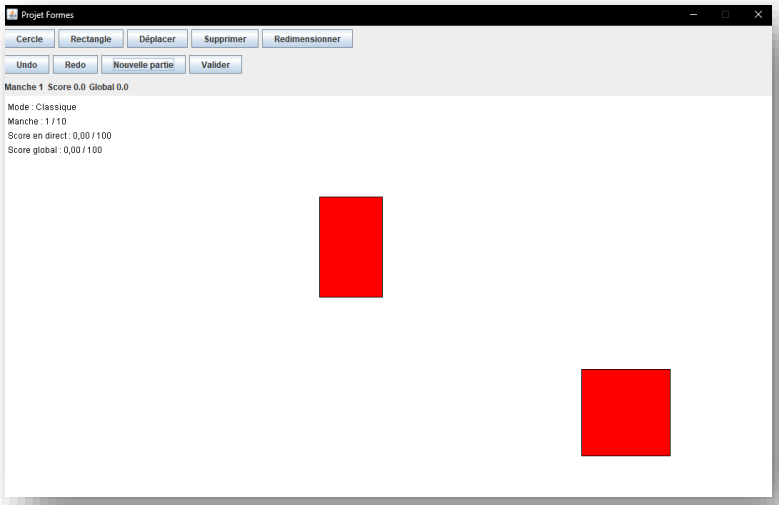
ANNEXE

Diagram de class : [version en ligne]

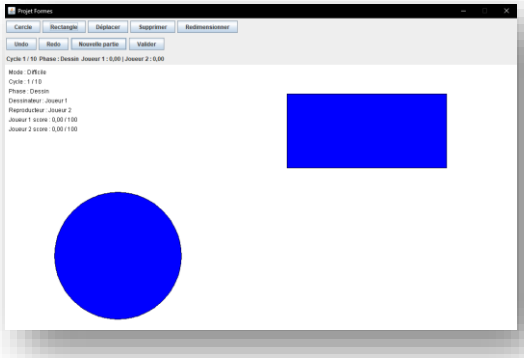
Nicolas
Dodart

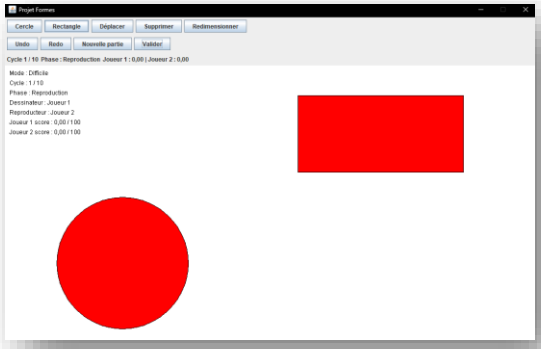
22409199
M1 INFO-IA-SD-S

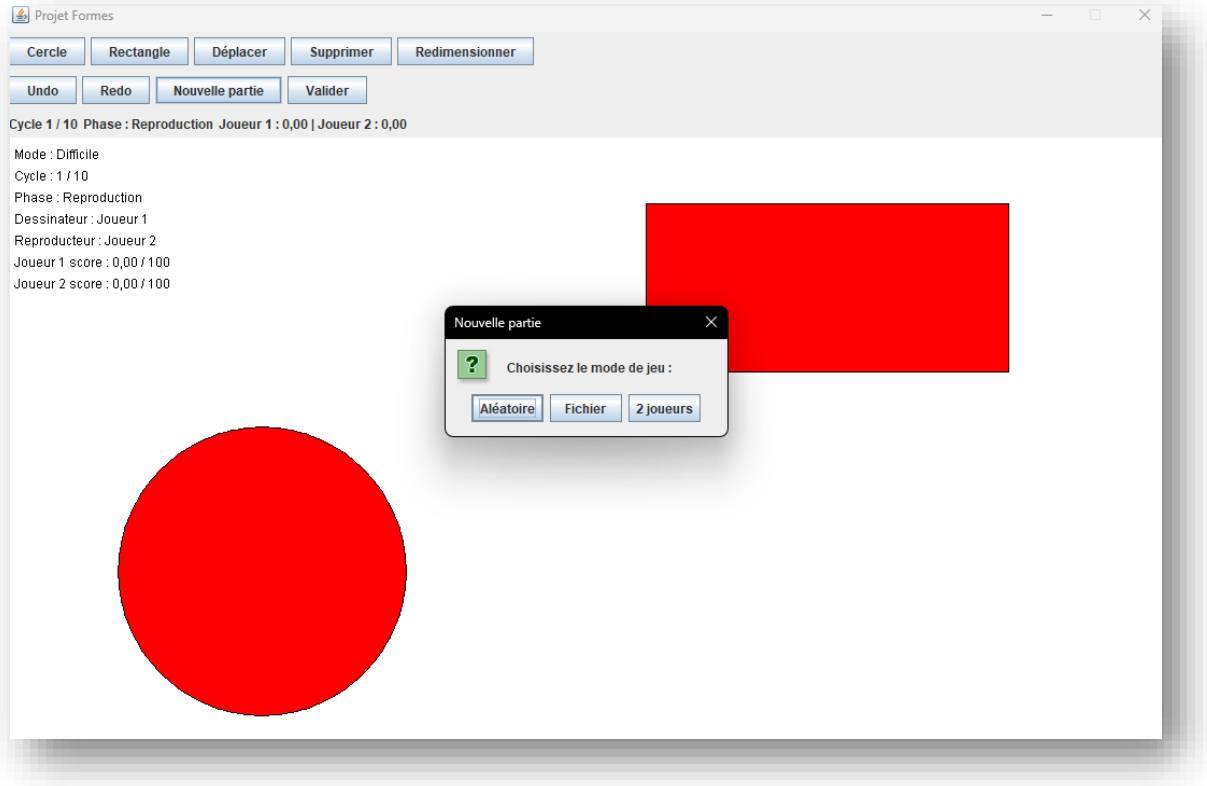
Ui de base :

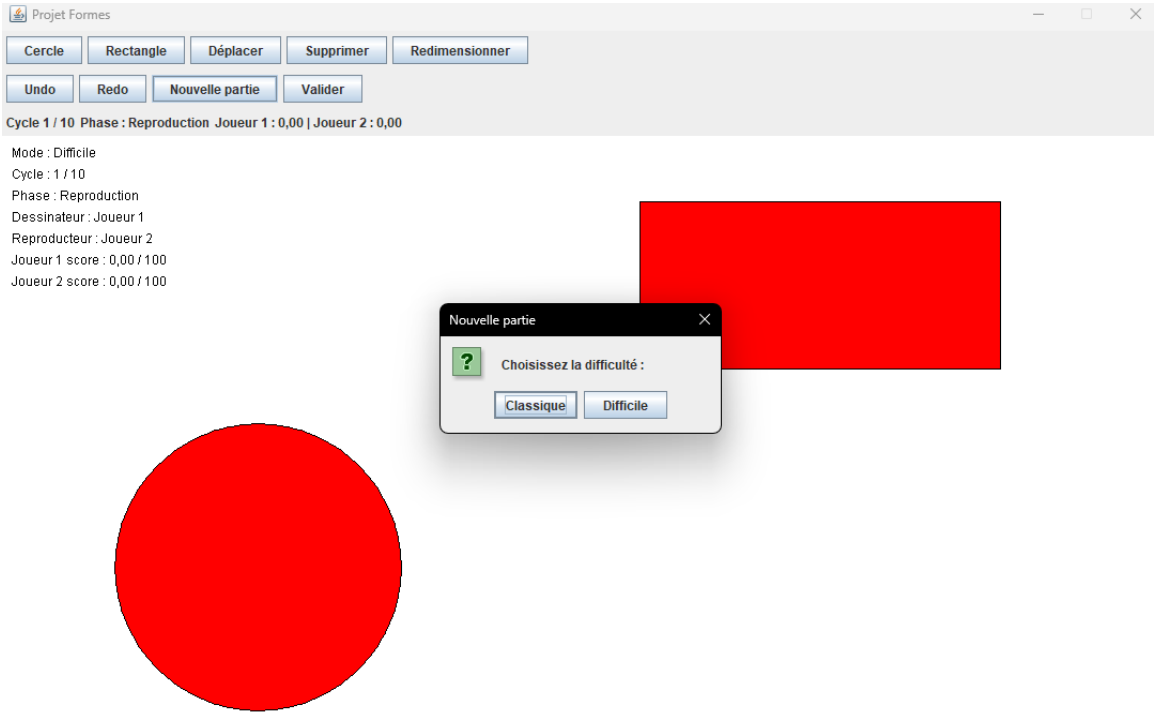


Mode 2 joueurs :






Projet Formes
Cercle
Rectangle
Déplacer
Supprimer
Redimensionner
Undo
Redo
Nouvelle partie
Valider
Cycle 1 / 10 Phase : Reproduction Joueur 1 : 0,00 | Joueur 2 : 0,00
Mode : Difficile
Cycle : 1 / 10
Phase : Reproduction
Dessinateur : Joueur 1
Reproducteur : Joueur 2
Joueur 1 score : 0,00 / 100
Joueur 2 score : 0,00 / 100
Nouvelle partie
Choisissez le mode de jeu :
Aléatoire
Fichier
2 joueurs


Projet Formes
Cercle
Rectangle
Déplacer
Supprimer
Redimensionner
Undo
Redo
Nouvelle partie
Valider
Cycle 1 / 10 Phase : Reproduction Joueur 1 : 0,00 | Joueur 2 : 0,00
Mode : Difficile
Cycle : 1 / 10
Phase : Reproduction
Dessinateur : Joueur 1
Reproducteur : Joueur 2
Joueur 1 score : 0,00 / 100
Joueur 2 score : 0,00 / 100
Nouvelle partie
Choisissez la difficulté :
Classique
Difficile